

Scaling Fermionic Simulations with Quantum-Centric Supercomputing

The scientific problem does not need to **fit on the quantum computer**.

The quantum computer **needs to fit into the scientific workflow**.

Thaddeus Pellegrini, Quantum Algorithms Researcher

IBM Quantum · IEEE Quantum Week 2026

Table of Contents

- 1. From compression to decomposition**
How fermionic simulations evolved from fitting problems onto QPUs to decomposing large scientific systems.
- 2. Embedded wavefunctions and heterogeneous solver routing**
EWF, fragment construction, classical/quantum solver assignment, and reconstruction.
- 3. Scaling the workflow**
TrimSQD, localized EWF, GPU diagonalization, and protein-scale execution.
- 4. Hands-on example: Glycine with Quantum Fragment Methods**
Full-system CCSD → EWF+CCSD → EWF+FCI/CCSD/SQD → TrimSQD.
- 5. QRMI and quantum resource management**
Resource lifecycle, configuration, core API, and Slurm/GRES integration.

Github Link to the tutorial
that we cover today



Where we started

From compression to decomposition

- **Scientific Target:** calculate the activation barrier of a chemically meaningful Diels-Alder Reaction
- **Computational Strategy:** Solve a selected active-space problem on quantum hardware, with classical post-processing to recover missing correlation
- **Hardware Strategy:** Use entanglement forging to compress the active-space representation enough to fit available QPU resources, with QSE improving the approximate solution

Key Historical Idea: Choose an interesting quantum subproblem, then compress it until it fits the best quantum device available.

[1] I. Liepuoniute *et al.*, “Simulation of a Diels–Alder reaction on a quantum computer,” *Phys. Chem. Chem. Phys.*, vol. 26, pp. 25181–25191, 2024, doi: 10.1039/D4CP01314J.

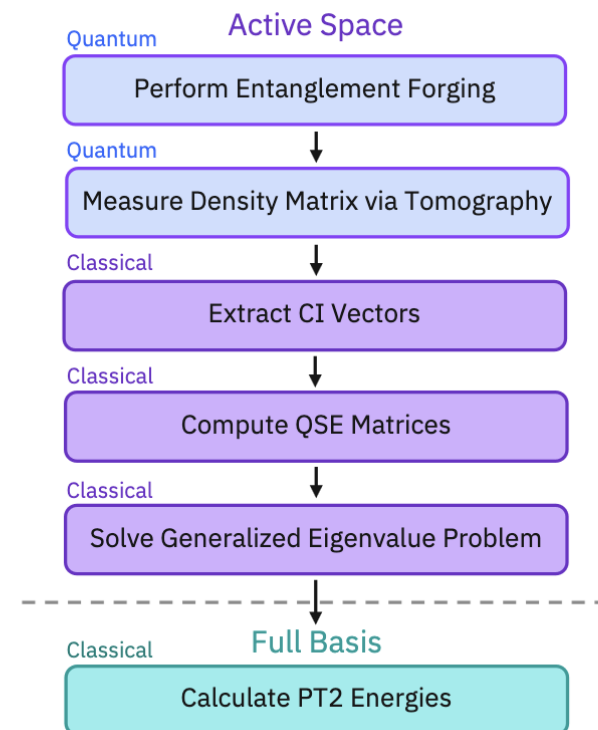


Figure 3. Schematic representation of the hybrid approach workflow. Active-space calculations are performed on a quantum computer, followed by classical post-processing. After conducting an active-space variational calculation with entanglement forging (EF), we conduct an active-space quantum subspace expansion calculation to refine the EF results and second-order perturbation theory (PT2) to account for dynamical electron correlation. To quantify and mitigate the errors occurring on quantum devices, we extract the density operators of the quantum states prepared by the device using tomography. We then project the density operator in the subspace of the Hilbert space with appropriate particle number and spin. We use the resulting projection (referred to as a CI vector) as a starting point for the QSE and PT2 calculations in lieu of the original density operator.

Decompose and Distribute

From a protein to 303 correlated calculations

- Demonstrated a large-scale heterogeneous quantum-classical workflow built around protein fragmentation.
- Inverted the earlier "select & compress" strategy to "decompose & distribute"
- This shift is necessary because biologically relevant systems are inherently large
- Beyond HF, DFT, and MP2, first-principles correlated electronic structure methods remain difficult to scale to proteins

select & compress → decompose & distribute

[2] Shajan *et al.*, J. Chem. Theory Comput. 22, 6041 (2026).

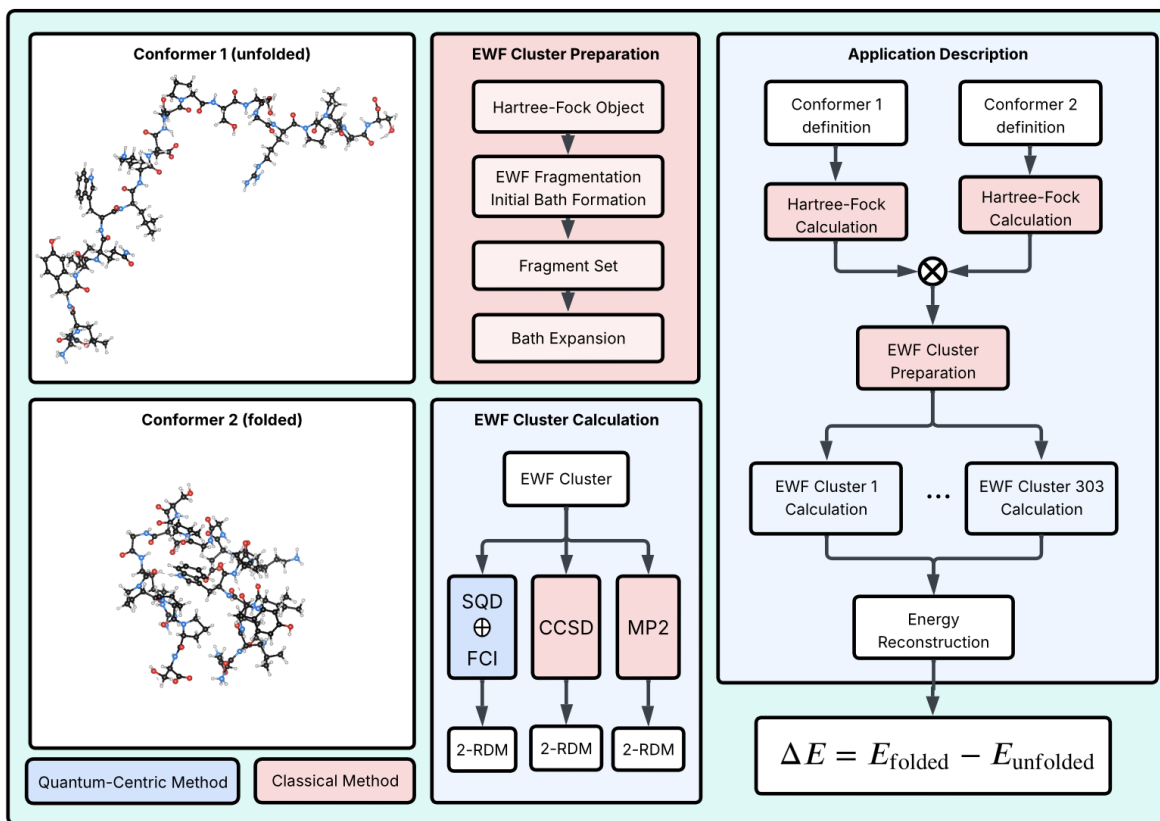


Figure 1: Overview of the EWF-based quantum-centric workflow for Trp-Cage conformer energetics. The protocol includes Hartree-Fock reference generation, automated fragmentation and bath construction, bath orbital expansion, and high-level cluster calculations using either quantum (SQR/FCI) or classical (MP2/CCSD) solvers. Fragment energies are reconstructed to obtain total conformer energies.

Embedded Wave Function

Constructing the Correlated Subproblem

decompose & distribute

Construction and correlation of the EWF clusters proceeds in two main steps:

1. Fragments are defined from localized orbitals and their mean-field entanglement baths
2. The minimal DMET bath is systematically expanded using correlated information to improve fragment-environment entanglement

[2] Shajan *et al.*, J. Chem. Theory Comput. 22, 6041 (2026).

[3] Nusspickel and Booth, *Phys. Rev. X* 12, 011046 (2022).

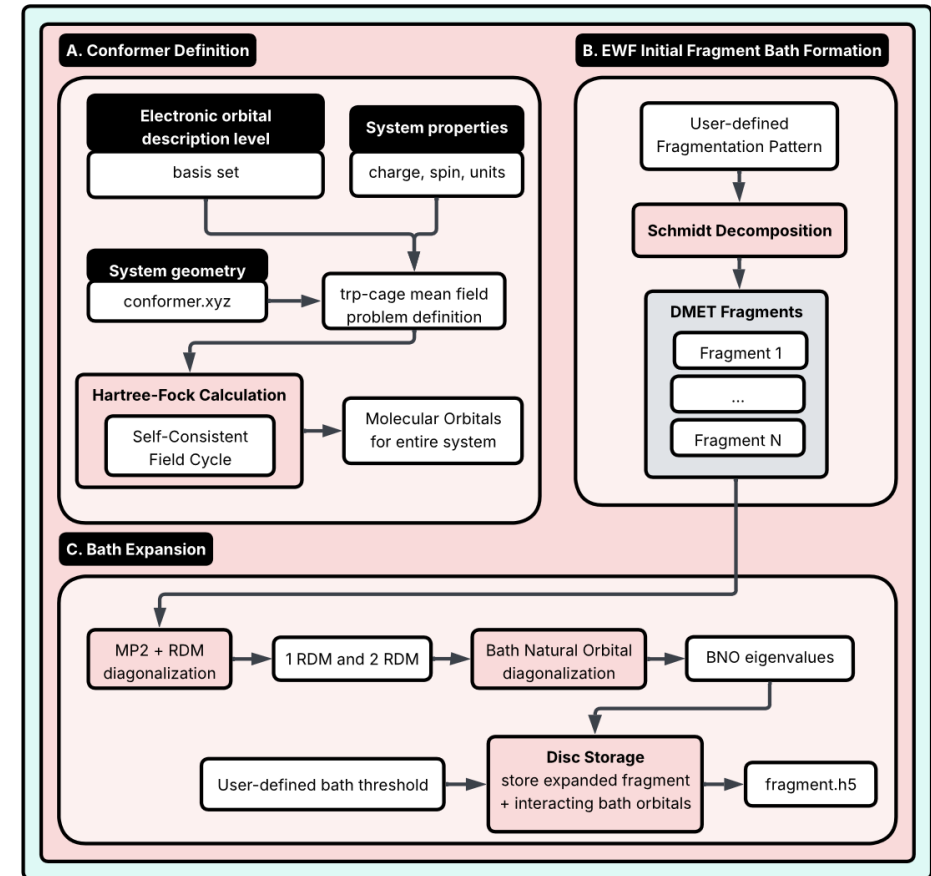


Figure 2: Schematic overview of the classical pre-processing and EWF workflow. The diagram illustrates (A) conformer definition and mean-field problem setup, (B) initial EWF fragment and DMET bath construction from the Hartree-Fock reference, and (C) MP2-level interacting bath expansion via bath natural orbitals.

Route each fragment to the right solver

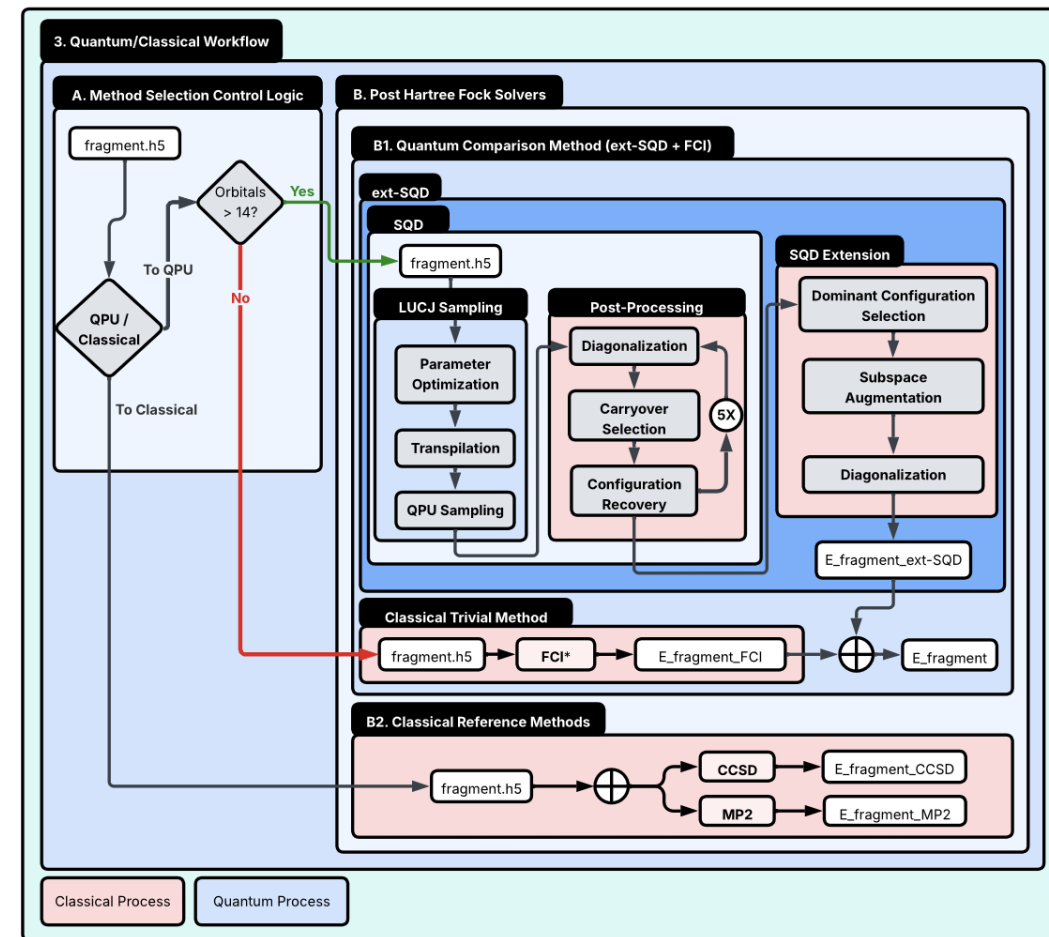
decompose & distribute

Quantum and classical resources become interchangeable components of the same embedded workflow.

Each fragment receives a correlated treatment:

1. Fragments are classified complexity, here by number of Molecular Orbitals
 1. Hard fragments → ext-SQD with LUCJ ansatz on QPU
 2. Simple fragments → FCI on classical hardware
2. CCSD and MP2 provide classical reference

Embedding creates the subproblems; solver routing decides where each subproblem should run



[2] Shajan *et al.*, J. Chem. Theory Comput. 22, 6041 (2026).

Figure 3: Quantum-classical workflow for post-Hartree-Fock EWF cluster calculations. The diagram summarizes solver selection logic, quantum circuit execution, and classical post-processing steps used in EWF-(SQD,FCI) calculations.

Can the workflow recover the conformer energy difference?

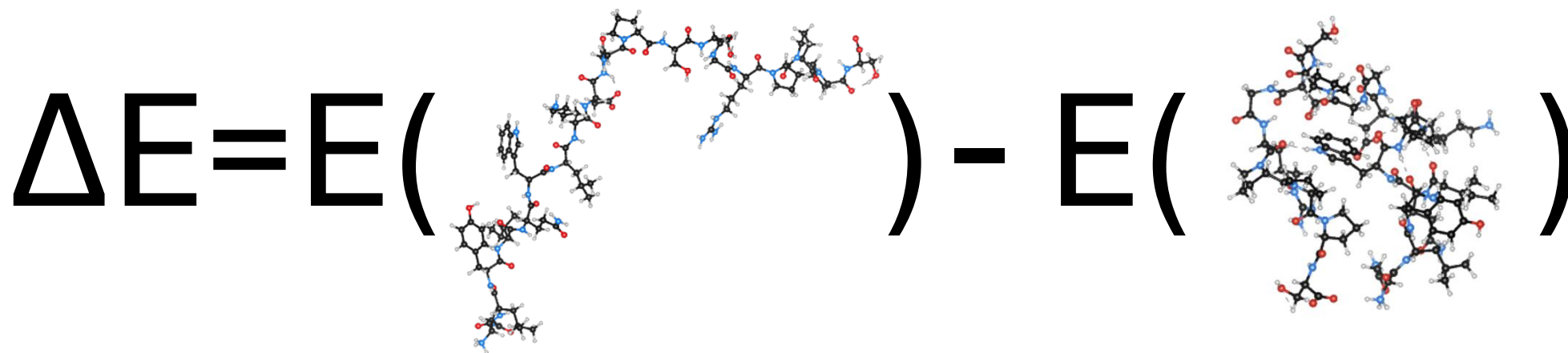


Table 1. Ground-State Energy of the TRP Cage in the Unfolded and Folded Conformations (Second, Third Column) and Relative Energy (Fourth Column) from Several Methods (Rows)

method	$E_{\text{unfolded}} (E_h)$	$E_{\text{folded}} (E_h)$	ΔE (kcal/mol)
RHF	-7343.9621	-7344.0400	48.90
RI-MP2	-7351.9420	-7352.0431	63.42
EWF-MP2	-7352.7377	-7352.8327	59.61 ($\Delta\Delta E_{\text{RI-MP2-EWF-MP2}} = 3.8$)
DLPNO-CCSD	-7353.5445	-7353.6275	52.05
EWF-CCSD	-7354.0653	-7354.1413	47.70 ($\Delta\Delta E_{\text{DLPNO-CCSD-EWF-CCSD}} = 4.4$)
EWF-(FCI,SQD)	-7354.1372	-7354.2256	55.43 ($\Delta\Delta E_{\text{DLPNO-CCSD-EWF-SQD}} = -3.4$)

303 embedded correlated calculations reconstruct a protein-scale relative energy competitive with high-level classical references

Does the workflow survive different chemistry?

Same architecture, different regime: apply EWF + heterogeneous solver routing to molten-salt chemistry instead of proteins

Why FLiBe matters: candidate fusion blanket material for tritium breeding and recovery

Why it is hard: strongly ionic and polarizable chemistry makes correlated electronic structure challenging

Scientific target: accurate tritium binding energies as inputs to free-energy calculations

Can present day quantum hardware solve the embedded electronic-structure problems needed for tritium chemistry?

[3] Das *et al.*, arXiv:2606.30402 (2026).

IBM Quantum

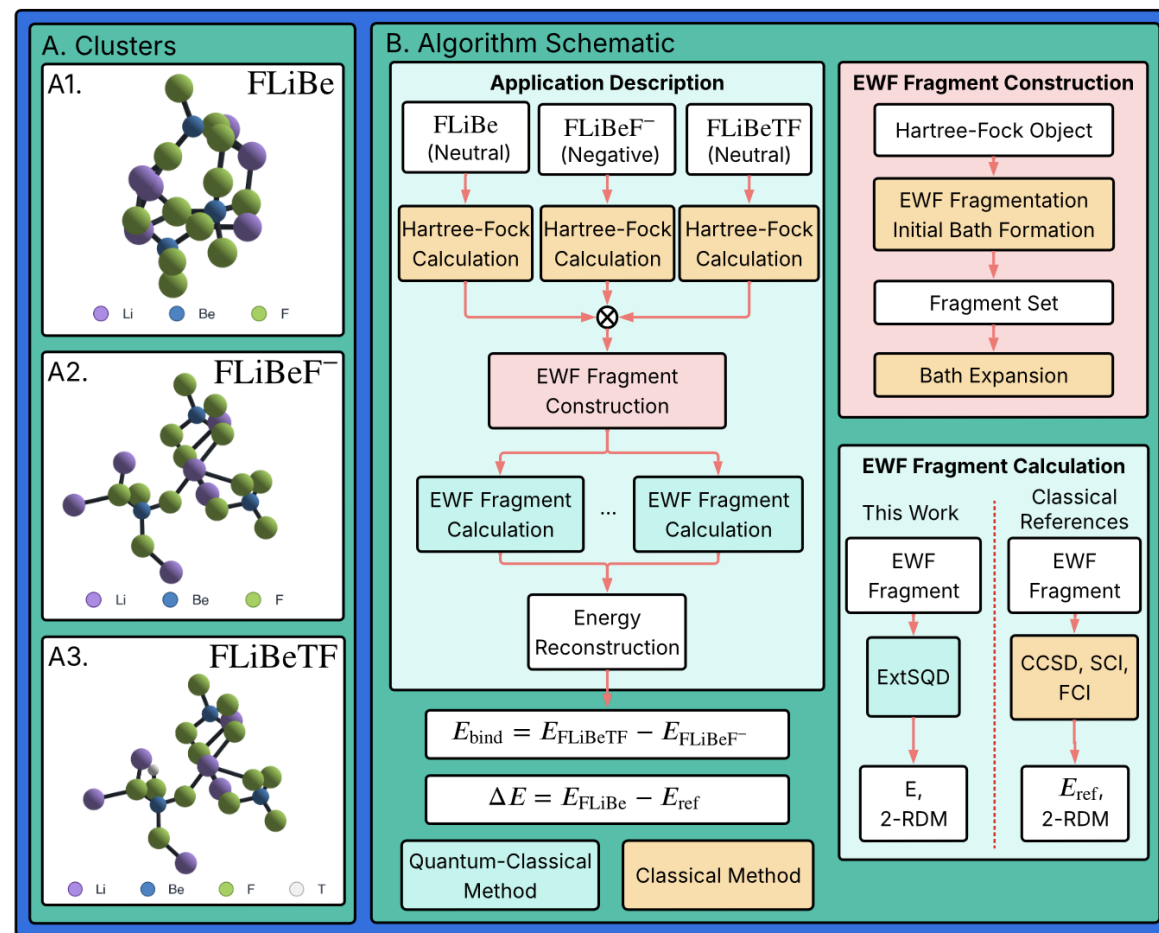


Figure 1: High-level overview of the heterogeneous quantum-classical embedded-wavefunction (EWF) workflow used in this work. (A) Representative cluster conformations: (A1) 21-atom $\text{Li}_6\text{Be}_3\text{F}_{12}$ (FLiBe), (A2) 22-atom $[\text{Li}_6\text{Be}_3\text{F}_{13}]^-$ (FLiBe⁻), and (A3) 23-atom $\text{Li}_6\text{Be}_3\text{F}_{13}\text{T}$ (FLiBe+T). Color code: Li (violet), Be (blue), F (green), T (grey). (B) Each conformation of each cluster is described by a Hartree-Fock calculation that feeds the EWF fragment construction, in which the Hartree-Fock object is partitioned into atom-centered fragments and a fragment Hamiltonian is built for each through localized-orbital fragmentation and localized two-electron-integral construction. Every fragment is then solved either by the quantum-classical route discussed in Fig. 2, extended sample-based quantum diagonalization (ext-SQD), or by classical reference solvers (CCSD, truncated CI, and FCI); each solver returns a fragment energy and two-body reduced density matrix, which are recombined in the energy-reconstruction step to give the total energy. For the tritium-binding application the target observable is $E_{\text{bind}} = E_{\text{FLiBeT}} - E_{\text{FLiBe}^-}$.

Where is the error coming from?

- Two methodological families
 1. **Fragmented:** same EWF construction, different correlated fragment solvers (quantum and classical)
 2. **Unfragmented:** full-system classical solvers

Within each family: results agree closely

Between families: a much larger gap appears

Therefore, the dominant bias comes from fragment construction / embedding, not the fragment solver

Solver error ≤ 0.7 kcal / mol

Embedding bias: ~ 10 -30 kcal / mol

The quantum solver is already accurate enough; improving the scientific result now requires improving the embedding

[3] Das *et al.*, arXiv:2606.30402 (2026).

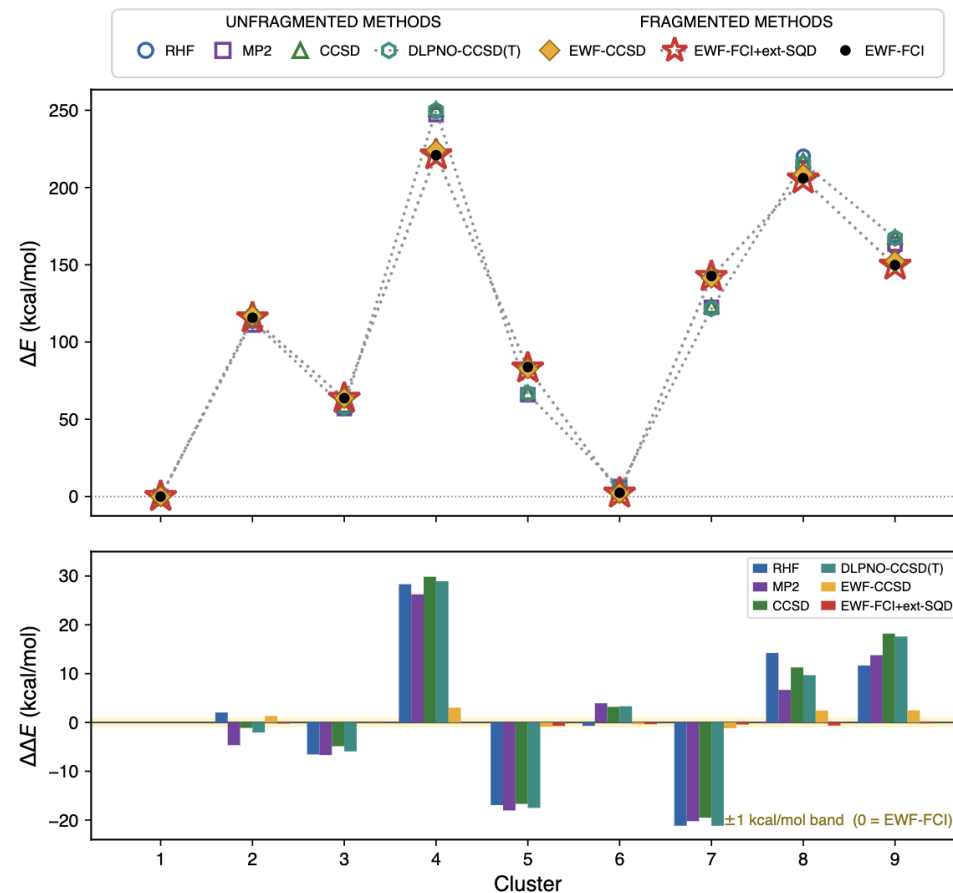


Figure 5: Top panel: relative energy $\Delta E = E_R - E_{R_1}$ (kcal/mol) of nine 21-atom $\text{Li}_6\text{Be}_3\text{F}_{12}$ clusters, each referenced to cluster 1. The four full-molecule (unfragmented) methods (RHF, MP2, DLPNO-CCSD(T), and canonical CCSD) are drawn as open markers and grouped at the left of the legend; the three embedded (fragmented) methods (EWF-CCSD, drawn as a filled diamond, EWF-FCI+ext-SQD, and EWF-FCI) at the right. The quantum-hardware result EWF-FCI+ext-SQD is drawn as a large open red star and EWF-FCI as a small filled black circle. Dotted lines connect the nine EWF-FCI+ext-SQD values and the nine DLPNO-CCSD(T) values across clusters, and the vertical separation between these two trends marks the embedding gap between the best full-molecule and embedded references. Bottom panel: deviation of each method from the EWF-FCI reference, $\Delta\Delta E = \Delta E_{\text{method}} - \Delta E_{\text{EWF-FCI}}$, drawn as grouped bars per cluster in the legend colours (unfragmented methods together with the fragmented EWF-CCSD and EWF-FCI+ext-SQD); the ± 1 kcal/mol accuracy window is shaded in gold and the zero line is the EWF-FCI reference. The unfragmented bars deviate by up to ~ 30 kcal/mol, whereas the fragmented bars remain within a few kcal/mol of EWF-FCI, the ext-SQD deviations lying inside the gold band.

Why use a Quantum Solver at All?

- Fragmentation solves one scaling problem:
 - Make the global system tractable
- But it *does not* solve another:
 - A single embedded fragment can still have a combinatorially hard correlated wavefunction

Embedding reduces the global size.

Quantum solvers target local electronic complexity

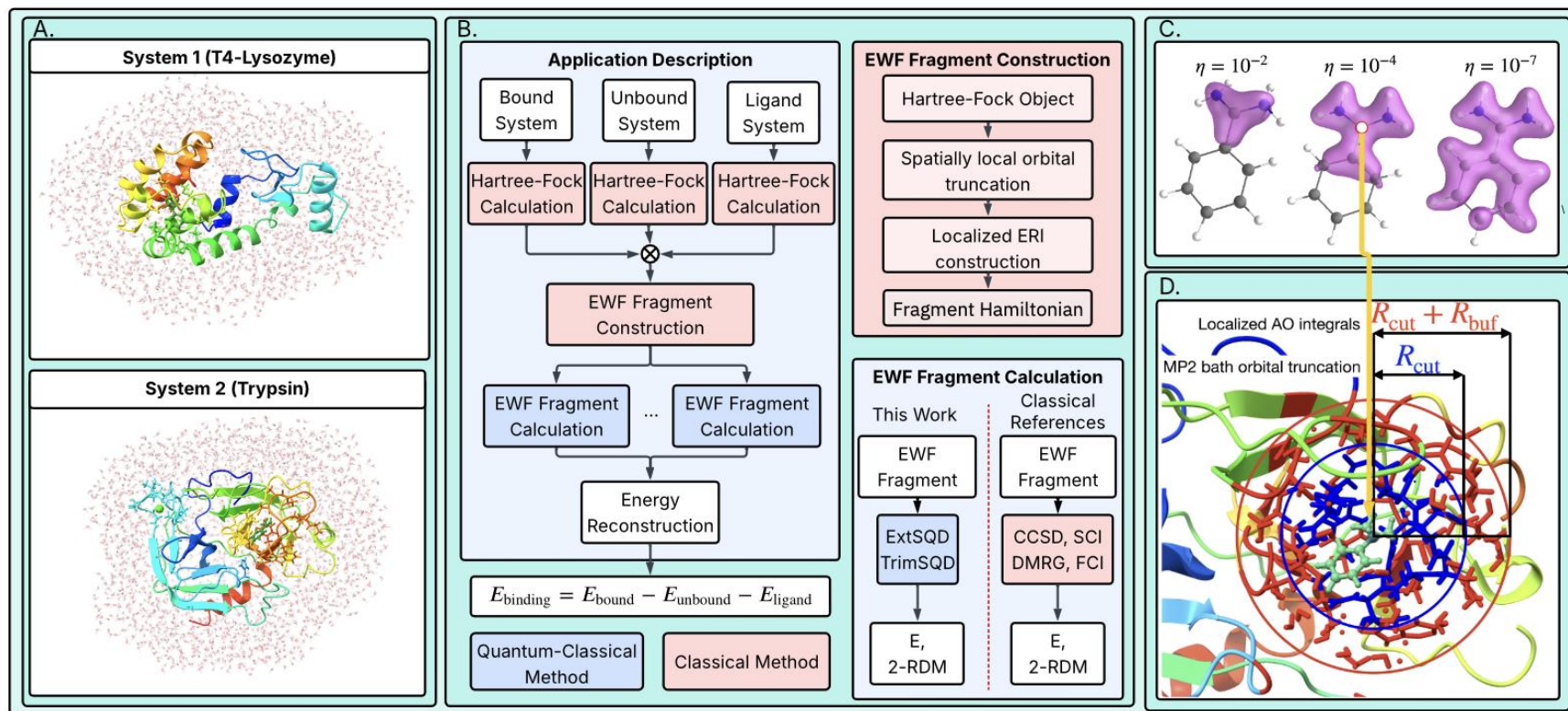
1. Classical solvers remain preferable for easy fragments
2. Exact classical CI scales poorly as fragment complexity grows
3. SQD is being validated as a solver that can occupy the same fragment-solver slot with comparable accuracy

The goal is not to use the QPU everywhere – only where the fragment becomes the bottleneck.

Scale the workflow

303 atoms → 12,635 atoms

>40x larger system
Trp-cage: 303 atoms
Trypsin: 12,635 atoms



Quantum workload

9,200 circuits
Up to 94 qubits
1.3e9 measurements

Heterogeneous resources

2 Heron r2 QPUs
Fugaku + Miyabi-G

Distributed execution

> 100 QPU hours
72.5% parallel efficiency

At this scale, electronic-structure becomes a distributed systems problem as much as a quantum-algorithm problem

Remove the Bottlenecks

Localized EWF + TrimSQD + GPU

Diagonalization

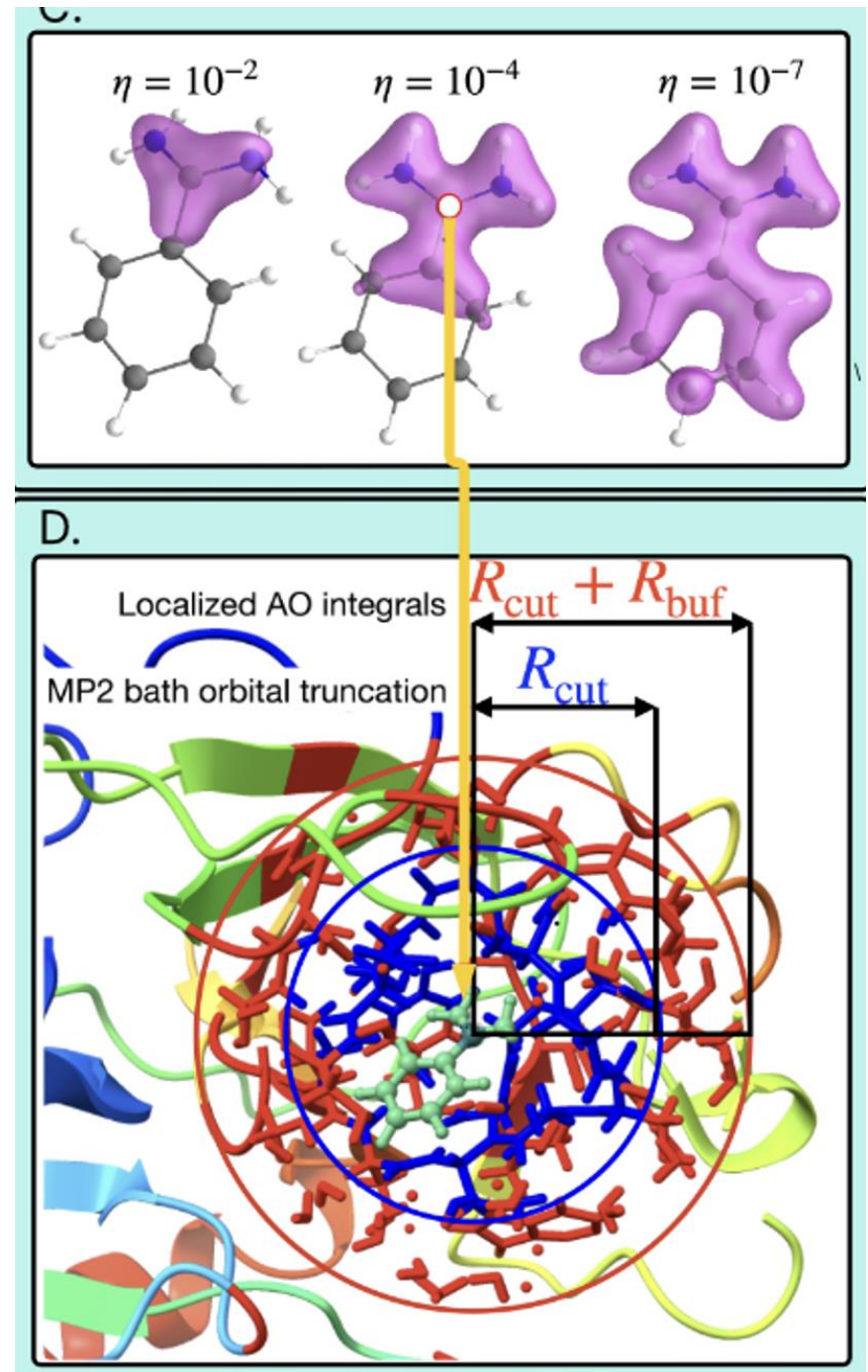
Fragment construction was too global

Bottleneck:

- Conventional EWF construction retained full-system dependencies
- Memory/runtime grew too steeply with system size

Fix:

- Spatially localized orbital truncation
- Localized ERI construction
- Reduces fragment constrain from $\sim O(M) \rightarrow O(1)$ **per fragment**
- Makes fragment construction embarrassingly parallel



Remove the Bottlenecks

Localized EWF + **TrimSQD** + **GPU**

Diagonalization

Fragment solving was too expensive

Bottleneck:

- Ext-SQD subspace diagonalization dominated runtime
- Accuracy degraded for the hardest/largest fragments

Fix:

- TrimSQD improves determinant selection
- GPU-accelerated selected-basis diagonalization
- Distributed-memory execution across many GPU nodes

TrimSQD → better configuration selection
SBD-G → scalable GPU diagonalization

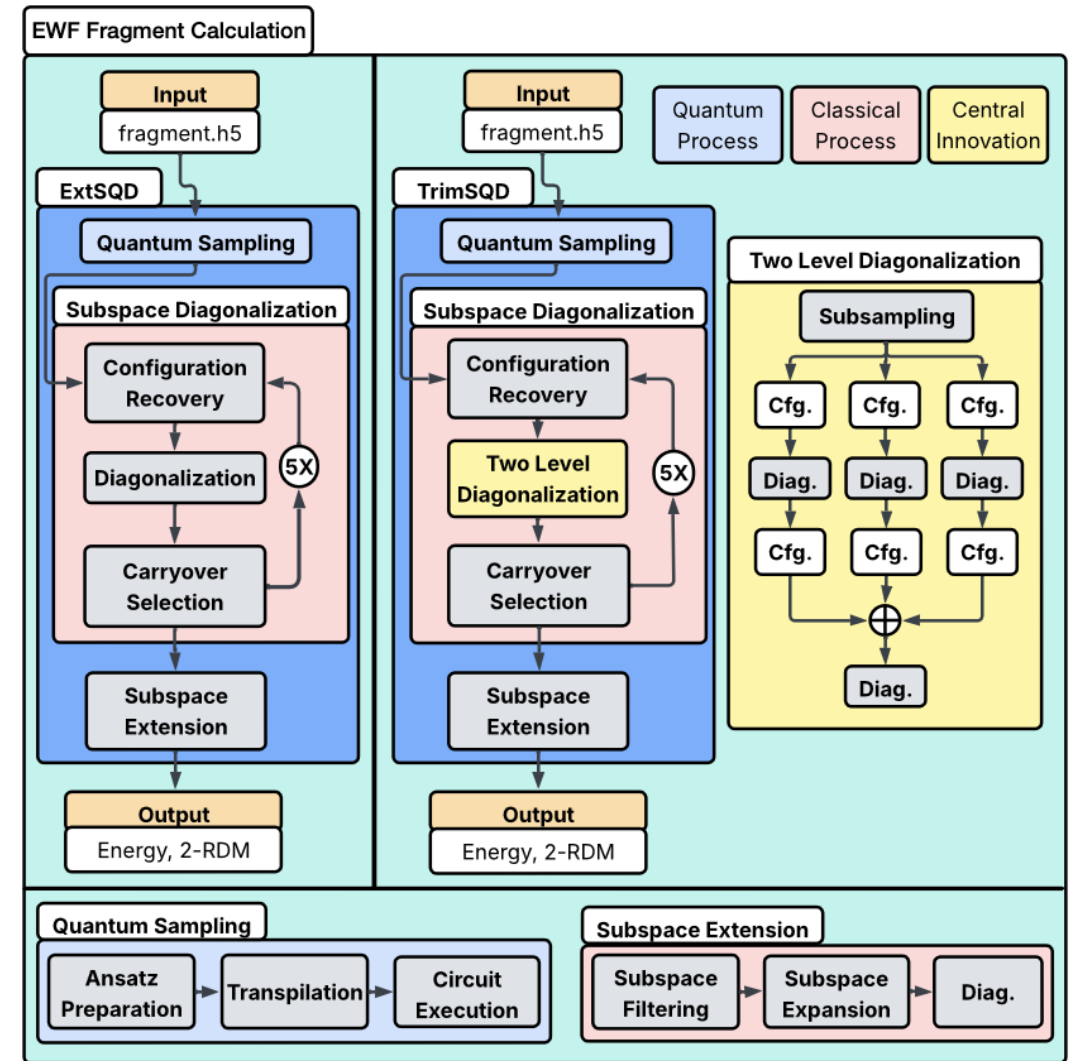


Fig. 2. The workflow of ExtSQD (left) and TrimSQD (middle), with central innovations from this study marked in light yellow (right). “Cfg.” is an abbreviation for “electronic configurations”, x in the main text.

[4] Merz *et al.*, arXiv:2605.01138 (2026).

Quantum solver gains grow with fragment difficulty

- **Small CI spaces:** TrimSQR and ExtSQR are similar
- **Larger CI spaces:** TrimSQR produces progressively lower fragment energies
- **Hardest tested fragmentations:** improvement is largest where CI space approaches 10^{18} - 10^{21}

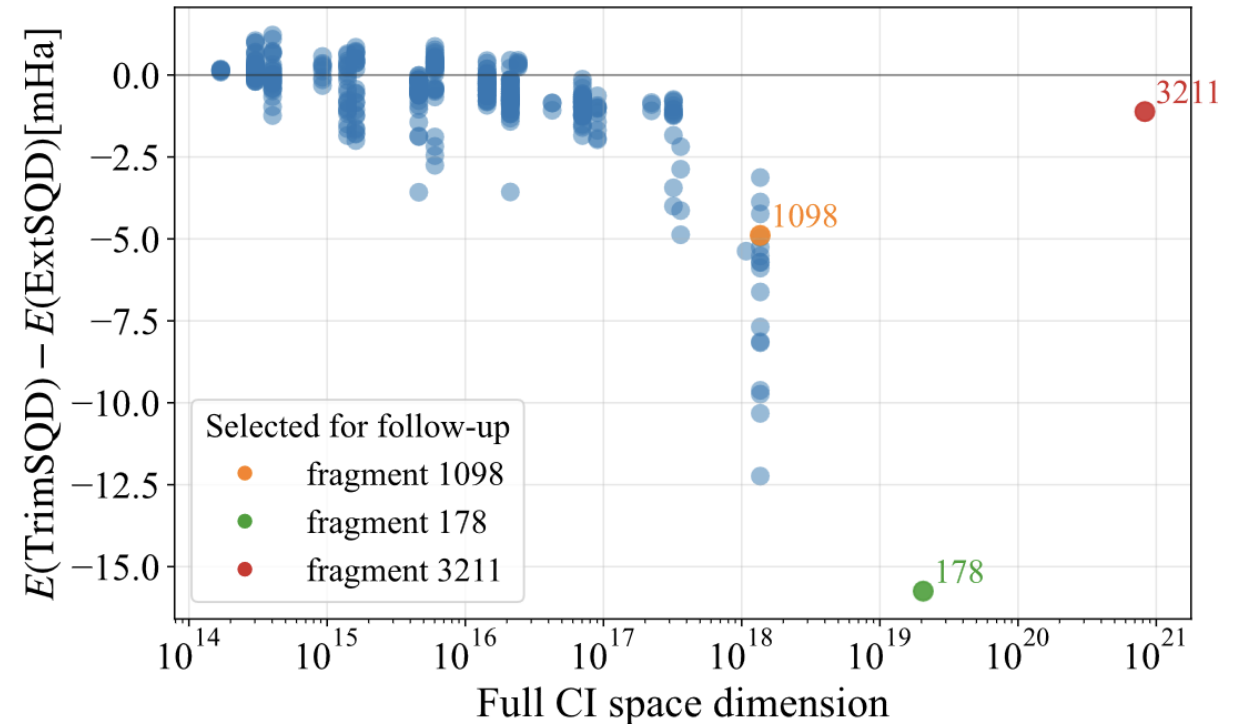
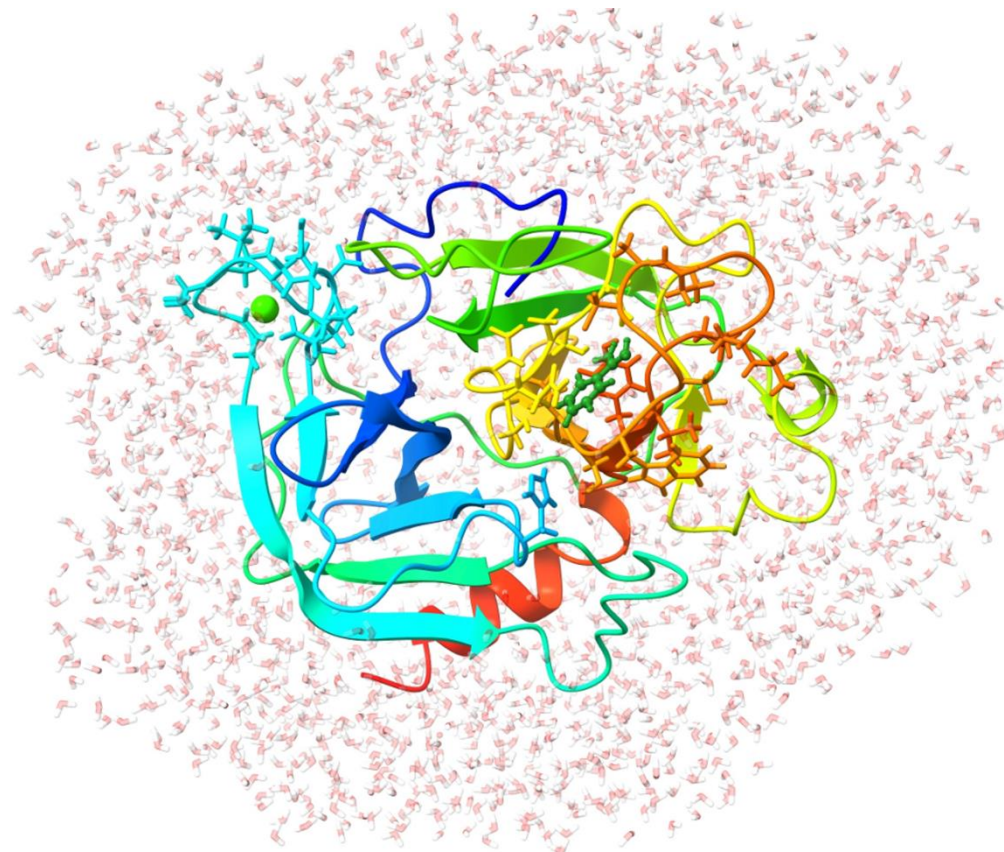


Fig. 5. Accuracy improvement of TrimSQR over ExtSQR.

Preserve the chemistry

>40x larger systems + ~210x better
fragment accuracy



Trp-Cage / EWF-ExtSQD

303 Atoms

Largest fragment: ~ 34e, 33o

Largest fragment HQC error: **21 mHa**

Trypsin / Local EWF-TrimSQD

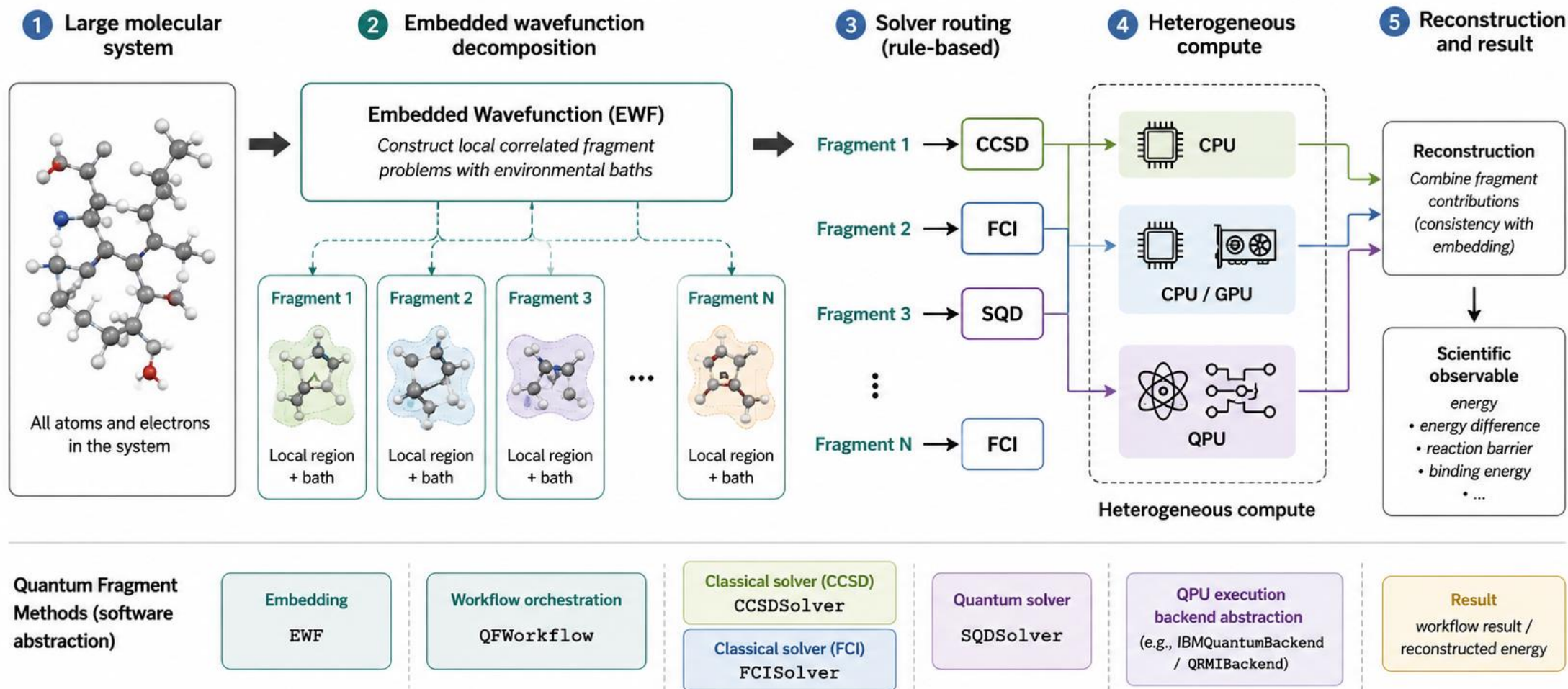
12,635 atoms

Largest fragment: ~ 66e, 45o

Largest fragment HQC error: **0.10 mHa**

**The workflow scaled in system size and improved
in solver accuracy at the same time**

From Scientific Problem to Heterogeneous Workflow



Embedding turns large-scale electronic structure into a schedulable collection of classical and quantum fragment calculations.

Quantum Fragment Methods

Qiskit Community Repo – Follow Along

Tutorials

- **Laptop:** CPU- Only + CPU/QPU
- **HPC:** CPU+GPU+QPU
- **Fully containerized**
- **Open Source**



Next: reproduce the architecture on glycine, the simplest amino acid

[5] T. Pellegrini *et al.*, *Quantum Fragment Methods*, v0.0.1 (2026), GitHub.

[README](#) [Code of conduct](#) [Contributing](#) [License](#)

quantum-fragment-methods

License [Apache 2.0](#) python [3.8+](#) code style [black](#) status [alpha](#)

About

A Python package for quantum chemistry tailored toward fragment-based embedding and quantum algorithms applied to macromolecule simulations. This codebase provides open access to the methodology from the paper "*Molecular Quantum Computations on a Protein*", a collaborative effort between IBM Quantum and the Cleveland Clinic Foundation.

Overview

The Quantum Fragment Methods framework enables scalable, high-accuracy quantum simulations of large molecular systems — such as peptides and proteins — by embedding them into quantum subspaces. The package features:

- **Embedder objects:** EWF with configurable bath and truncation parameters
- **Rule-based solver assignment:** Priority-based automatic solver selection per fragment
- **Quantum solvers:** SQD (production), ext-SQD (planned)
- **Classical solvers:** FCI, CCSD

Documentation

- [Local Build Guide](#) — laptop / workstation setup with Podman or Docker
- [HPC Build Guide](#) — cluster deploy (Slurm, transferring images, native SBD/PyCI)
- [Tutorial](#) — interactive Jupyter notebook with usage examples
- [Contributing Guide](#) — guidelines for contributors
- [Examples](#) — additional example guides

✓ EWF + SQD Demo — Glycine (Simplest Amino Acid)

This notebook demonstrates four computational approaches for molecular quantum calculations on **glycine** ($\text{H}_2\text{N}-\text{CH}_2-\text{COOH}$), the simplest amino acid. It replicates the methodology from *"Molecular Quantum Computation on a Protein"* at a scale that runs comfortably on a laptop — with one real QPU fragment.

Method	Embedding	Solver(s)	QPU?
1	None	Full-system CCSD	No
2	EWF	CCSD per fragment	No
3	EWF	FCI (small frags) + SQD (C=O fragment)	Yes
4	EWF	FCI (small frags) + TrimSQD (C=O fragment)	Yes (reused)

Methods 3 and 4 use the **same QPU job** — TrimSQD re-uses the `counts.npy` checkpoint and applies a screening-based classical post-processing policy that typically recovers lower energies with the same shot budget.

Why glycine?

Glycine has 5 heavy atoms (N, C α , C=O, O $\times$ 2). IAO fragmentation with an MP2 bath at `truncation=1e-4` produces 10 fragments. The C=O carbonyl fragment grows to 19 cluster orbitals — above the classical threshold — making it the natural candidate for SQD. All other fragments run as FCI or CCSD in seconds.

Fragment map (STO-3G, three-tier adaptive)

Frag 2	C=O fragment	19 orb	→ SQD / TrimSQD on ibm_kingston (38 qubits)
Frag 1	C α -fragment	~17 orb	→ CCSD
Frag 0	N-fragment	~13 orb	→ CCSD
Frag 3	O-fragment	~13 orb	→ CCSD
Frag 4	O-fragment	~13 orb	→ CCSD
Frag 5-9	H $\times$ 5	3-4 orb	→ FCI



Configuration inputs from `config.yaml`

Inputs are defined in the system specific configuration file `config.yaml` based on the scientific workflow and researchers judgement



FOLLOW ALONG →

```
1 # Load config - all tunable parameters live here
2 with open(file=CONFIG_FILE) as f:
3     config = yaml.safe_load(stream=f)
4
5 qpu_config      = config["qpu"]
6 sqd_config      = config["sqd"]
7 embedder_config = config["embedder"]["ewf"]
8 sel_config      = config["solver_selection"]
9 fci_threshold   = sel_config["fci_threshold"]
10 ccsd_threshold  = sel_config["ccsd_threshold"]
11 sqd_threshold   = sel_config["sqd_threshold"]
12 sampler_opts    = qpu_config.get("sampler_options", {})
13
14 print(f"Basis           : {config['workflow']['basis']}")
15 print(f"Bath type       : {embedder_config['bath_type']}")
16 print(f"Truncation       : {embedder_config['truncation']}")
17 print(f"FCI threshold    : n_orb ≤ {fci_threshold}")
18 print(f"CCSD threshold    : {fci_threshold} < n_orb < {sqd_threshold}")
19 print(f"SQD threshold      : n_orb ≥ {sqd_threshold} (ibm_kingston)")
20 print(f"QPU backend        : {qpu_config['backend_name']}")
21 print(f"Shots              : {sampler_opts.get('default_shots', 10000)}")
22 dd = sampler_opts.get('dynamical_decoupling', {})
23 print(f"DD enabled          : {dd.get('enable', False)} (sequence: {dd.get('sequence_type', 'n/a')})")
24
```

[4] ✓ 0.0s Python

```
... Basis           : sto-3g
Bath type       : mp2
Truncation      : 0.0001
FCI threshold   : n_orb ≤ 10
CCSD threshold  : 10 < n_orb < 19
SQD threshold   : n_orb ≥ 19 (ibm_kingston)
QPU backend     : ibm_kingston
Shots           : 100000
DD enabled      : True (sequence: XY4)
```

+ Code + Markdown

Method 1 — Full-system CCSD (reference)

Single coupled-cluster calculation on the entire glycine molecule, no fragmentation. Serves as the accuracy benchmark.

- 10 atoms, STO-3G → 28 basis functions
- Wall time: ~5 s on a laptop

```
1 mol: Mole = pyscf.gto.Mole()
2 mol.atom = geometry_data
3 mol.unit = "Angstrom"
4 mol.basis = config["workflow"]["basis"]
5 mol.verbose = 3
6 mol.build()
```

[5] ✓ 0.0s

Python

... <pyscf.gto.mole.Mole at 0x120b7ce90>

```
1 mf: pyscf.scf.hf.RHF | pyscf.scf.hf_symm.RHF ... = scf.RHF(mol)
2 mf.kernel()
3 hf_energy: int | Any = mf.e_tot
4 print(f"HF energy: {hf_energy:.8f} Ha")
```

[6] ✓ 0.1s

Python

... converged SCF energy = -278.905788665211
HF energy: -278.90578867 Ha

Compute a full-system classical computation as a reference

We begin with a Restricted Hartree-Fock Calculation, the mean-field starting point of all methods

This establishes an approximate many-electron ground state and energy



FOLLOW ALONG →

Execute the full-system CCSD reference

CCSD Scales as $O(N^6)$ and becomes prohibitive for large molecular systems.

Glycine is small enough to solve without fragmentation, giving us a classical reference for the embedded workflows that follow.

```
1 mycc: pyscf.cc.dfuccsd.UCCSD | pyscf.cc.uccsd.U... = cc.CCSD(mf)
2 mycc.kernel()
3 energy_full_ccsd: Any = mycc.e_tot
4
5 print(f"\n{'='*55}")
6 print(f"Method 1: Full-system CCSD")
7 print(f"{'='*55}")
8 print(f"HF energy      : {hf_energy:.8f} Ha")
9 print(f"CCSD energy     : {energy_full_ccsd:.8f} Ha")
10 print(f"Correlation    : {(energy_full_ccsd - hf_energy)*1000:.4f} m
11 print(f"{'='*55}")
```

[7] ✓ 0.2s Python

... E(CCSd) = -279.2162392572901 E_corr = -0.3104505920787068

```
=====
Method 1: Full-system CCSD
=====
```

```
HF energy      : -278.90578867 Ha
CCSD energy     : -279.21623926 Ha
Correlation    : -310.4506 mHa
=====
```

FOLLOW ALONG →



Method 2 — EWF + CCSD (fragmented, all classical)

The Embedded Wave Function (EWF) method partitions glycine into IAO fragments, adds an MP2 bath to each, then solves each fragment independently with CCSD. This is the classical baseline for the hybrid method below.

- Fragmentation: IAO (one fragment per heavy atom + H pseudo-fragments)
- Bath: MP2, `truncation=1e-4`
- Solver: CCSD for all fragments
- Wall time: ~30–60 s on a laptop

```
1 ewf_ccsd: EWF = EWF(  
2     bath_type = embedder_config["bath_type"],  
3     truncation = embedder_config["truncation"],  
4 )  
5  
6 workflow_ccsd: QFWorkflow = QFWorkflow(  
7     geometry      = geometry_data,  
8     basis         = config["workflow"]["basis"],  
9     embedder      = ewf_ccsd,  
10    fragmentation = embedder_config["fragmentation"],  
11    save_path      = str(DEMO_DIR / "results_ewf_ccsd"),  
12 )  
13  
14 workflow_ccsd.add_solver_rule(  
15     solver_factory = lambda frag: CCSD(),  
16     condition      = None,  
17     priority       = 0,  
18 )
```

3]

✓ 0.0s

Python

Construct the embedded workflow

Embedding object

Workflow orchestration

Solver assignment system

Correlated fragment solver



Executing the workflow across fragments

Each fragment is solved according to the workflow's routing rules

Fragment contributions are recombined through the EWD energy reconstruction

```
[9] 1 print("Running EWF+CCSD...")
    2 results_ewf_ccsd: WorkflowResult = workflow_ccsd.run()
    3 energy_ewf_ccsd: Any = results_ewf_ccsd.total_energy

Python

... Fragment 4 correlation energy: -0.12957824 Ha
Solving fragment 5 with CCSD...
converged SCF energy = -7.68463272182046
Fragment 5 correlation energy: -0.01691770 Ha
Solving fragment 6 with CCSD...
converged SCF energy = -7.64532334469819
Fragment 6 correlation energy: -0.01684391 Ha
Solving fragment 7 with CCSD...
converged SCF energy = -7.6453233446982
Fragment 7 correlation energy: -0.01684391 Ha
Solving fragment 8 with CCSD...
converged SCF energy = -6.03181421871182
Fragment 8 correlation energy: -0.01758380 Ha
Solving fragment 9 with CCSD...
converged SCF energy = -6.03181421871183
Fragment 9 correlation energy: -0.01758380 Ha

Partitioned Cumulant Energy Reconstruction:
Mean-field energy: -278.90622764 Ha
1-body PC correction: 0.34458617 Ha
2-body PC correction: -0.70963214 Ha
Total energy: -279.27127361 Ha

=====
Method 2: EWF + CCSD
=====
Fragments      : 10
Total energy   : -279.27127361 Ha
Δ from CCSD ref: -55.0344 mHa
=====
```



Method 3 — EWF + FCI/CCSD/SQD (adaptive, hybrid quantum)

Same EWF fragmentation, but now with **adaptive solver selection**:

Rule	Condition	Solver
Priority 20	<code>n_orbitals <= 11</code>	FCI (exact, fast)
Priority 10	<code>12 <= n_orbitals < 19</code>	CCSD (approx., fast)
Priority 0	<code>n_orbitals >= 19</code>	SQD on ibm_kingston

The C=O carbonyl fragment (19 orbitals) is the fragment that meets the SQD threshold. Its 38-qubit LUCJ circuit is submitted to `ibm_kingston`.

QPU workflow (checkpoint-aware):

1. Running this cell submits the QPU job and prints a job ID.
2. The job ID is saved to `results_ewf_adaptive/fragment_0/job_id.txt`.
3. Re-running after the job completes automatically resumes — no resubmission.
4. To force a fresh QPU submission: delete `job_id.txt` and `counts.npy` from that directory.



3a — Accessing the QPU with QRMI

QRMI reads credentials and resource configuration from environment variables, keeping secrets out of the notebook and repository.

Setup (one time per environment):

1. Copy the example env file to `.env` in this directory:

```
cp examples/notebook_demos/ewf_sqd_demo/.env.example \
  examples/notebook_demos/ewf_sqd_demo/.env
```

2. Open `.env` and fill in your IBM Quantum credentials:

```
ibm_kingston_QRMI_IBM_QCS_IAM_APIKEY=<your-iam-api-key>
ibm_kingston_QRMI_IBM_QCS_SERVICE_CRN=<your-crn>
```

The prefix (`ibm_kingston_`) must be **lowercase** and match `QRMI_JOB_QPU_RESOURCES` exactly. Your IAM API key and CRN are available at quantum.ibm.com.

The `.env` file is gitignored — it will never be committed.

HPC / Slurm: When running on SCC, skip the `.env` file entirely. The Slurm SPANK plugin injects all QRMI environment variables automatically when your job requests `--gres=qpu:1 --qpu=ibm_kingston`.



```
quantum-fragment-methods > examples > notebook_demos > ewf_sqd_demo > ⚙️ .env.example
```

```
1  # QRMI credentials for the glycine EWF+SQD notebook demo
2  #
3  # Copy this file to .env and fill in your values:
4  #   cp .env.example .env
5  #
6  # The .env file is gitignored – your credentials will never be committed.
7  # Do NOT add real credentials to this .env.example file.
8  #
9  # --- How to get these values ---
10 # IAM API key : https://quantum.ibm.com/account → "API keys" → "Create key"
11 # Service CRN : https://quantum.ibm.com/account → "Instances" → copy the CRN
12 #               Format: crn:v1:bluemix:public:quantum-computing:REGION:a/ACCOUNT:SERVICE_ID::
13 #
14 # NOTE: The backend name prefix must be lowercase and match QRMI_JOB_QPU_RESOURCES exactly.
15 #       e.g. if QRMI_JOB_QPU_RESOURCES=ibm_kingston, the prefix is ibm_kingston_
16 #
17 # --- Endpoint URLs (leave as-is for standard IBM Quantum Cloud) ---
18 ibm_kingston_QRMI_IBM_QCS_ENDPOINT=https://quantum.cloud.ibm.com/api/v1
19 ibm_kingston_QRMI_IBM_QCS_IAM_ENDPOINT=https://iam.cloud.ibm.com
20 #
21 # --- Fill these in ---
22 ibm_kingston_QRMI_IBM_QCS_IAM_APIKEY=<your-iam-api-key>
23 ibm_kingston_QRMI_IBM_QCS_SERVICE_CRN=<your-crn>
24 #
25 # --- Backend / resource selection (matches QRMI_JOB_QPU_RESOURCES) ---
26 # Leave as-is unless you want to target a different backend.
27 # On SCC/Slurm, this variable is injected automatically by the SPANK plugin.
28 QRMI_JOB_QPU_RESOURCES=ibm_kingston
29 QRMI_JOB_QPU_TYPES=ibm-quantum-compute-service
```

QRMI Environment Configuration for Local Use

Credentials and resource configuration live outside the notebook in environment variables, keeping secrets out of source code and version control



Binding the workflow to a QRMI resource

```
1 import os
2 from dotenv import load_dotenv
3 from quantum_fragment_methods.qpu.qrmi import QRMIBackend
4
5 # Load .env without overriding variables already set by Slurm SPANK plugin
6 load_dotenv(dotenv_path=DEMO_DIR / ".env", override=False)
7
8 # Fall back to ibm_kingston for interactive use if Slurm hasn't set the resource
9 BACKEND_NAME: str = os.environ.get("QRMI_JOB_QPU_RESOURCES", "ibm_kingston")
10 os.environ.setdefault(key="QRMI_JOB_QPU_RESOURCES", value=BACKEND_NAME)
11 os.environ.setdefault(key="QRMI_JOB_QPU_TYPES", value="ibm-quantum-compute-service")
12
13 # Initialize QRMI backend – credentials come from env vars, not from config
14 backend: QRMIBackend = QRMIBackend(config={"backend_name": BACKEND_NAME})
15 backend.initialize()
16 backend.get_backend()
17
18 props: Dict[str, Any] = backend.get_backend_properties()
19 print(f"Backend   : {props['backend_name']}")
20 print(f"Type      : {props['resource_type']}")
21 print(f"Env var    : QRMI_JOB_QPU_RESOURCES={BACKEND_NAME}")
```

[10] ✓ 6.8s

Python

```
... Backend   : ibm_kingston
Type      : ResourceType.IBMQuantumComputeService
Env var    : QRMI_JOB_QPU_RESOURCES=ibm_kingston
```

QRMI backend abstraction



3b — Build workflow and solve

```
1 ewf_adaptive: EWF = EWF(  
2     bath_type = embedder_config["bath_type"],  
3     truncation = embedder_config["truncation"],  
4 )  
5  
6 workflow_adaptive: QWorkflow = QWorkflow(  
7     geometry = geometry_data,  
8     basis = config["workflow"]["basis"],  
9     embedder = ewf_adaptive,  
10    fragmentation = embedder_config["fragmentation"],  
11    save_path = str(DEMO_DIR / "results_ewf_adaptive"),  
12 )  
13  
14 fci_threshold = sel_config['fci_threshold'] # n_orb <= this → FCI  
15 ccscd_threshold = sel_config['ccscd_threshold'] # fci_threshold < n_orb <= this → CCSD  
16 sqd_threshold = sel_config['sqd_threshold'] # n_orb >= this → SQD  
17  
18 # Rule 1 (priority 20): FCI for H fragments (≤ fci_threshold orbitals)  
19 workflow_adaptive.add_solver_rule(  
20     solver_factory = lambda frag: FCI(),  
21     condition = lambda frag: frag.n_orbitals <= fci_threshold,  
22     priority = 20,  
23 )  
24  
25 # Rule 2 (priority 10): CCSD for medium fragments (fci_threshold < n_orb < sqd_threshold)  
26 workflow_adaptive.add_solver_rule(  
27     solver_factory = lambda frag: CCSD(),  
28     condition = lambda frag: fci_threshold < frag.n_orbitals < sqd_threshold,  
29     priority = 10,  
30 )  
31  
32 # Rule 3 (priority 0): SQD for the largest fragment (≥ sqd_threshold orbitals)  
33 # workflow_path is set per-fragment inside solve() so each fragment  
34 # keeps its own job_id.txt / counts.py checkpoint.  
35 workflow_adaptive.add_solver_rule(  
36     solver_factory = lambda frag: SQDSolver(qpu_backend=backend, config=sqd_config),  
37     condition = None,  
38     priority = 0,  
39 )  
40
```

11] ✓ 0.0s

Python

Fragment cluster sizes (from EWF+CCSD run):

Fragment 0:	cluster_norb= 13	frag_nao=0	→ CCSD
Fragment 1:	cluster_norb= 17	frag_nao=0	→ CCSD
Fragment 2:	cluster_norb= 19	frag_nao=0	→ SQD
Fragment 3:	cluster_norb= 13	frag_nao=0	→ CCSD
Fragment 4:	cluster_norb= 13	frag_nao=0	→ CCSD
Fragment 5:	cluster_norb= 3	frag_nao=0	→ FCI
Fragment 6:	cluster_norb= 3	frag_nao=0	→ FCI
Fragment 7:	cluster_norb= 3	frag_nao=0	→ FCI
Fragment 8:	cluster_norb= 4	frag_nao=0	→ FCI
Fragment 9:	cluster_norb= 4	frag_nao=0	→ FCI

Priority based
solver routing

SQD parameters:

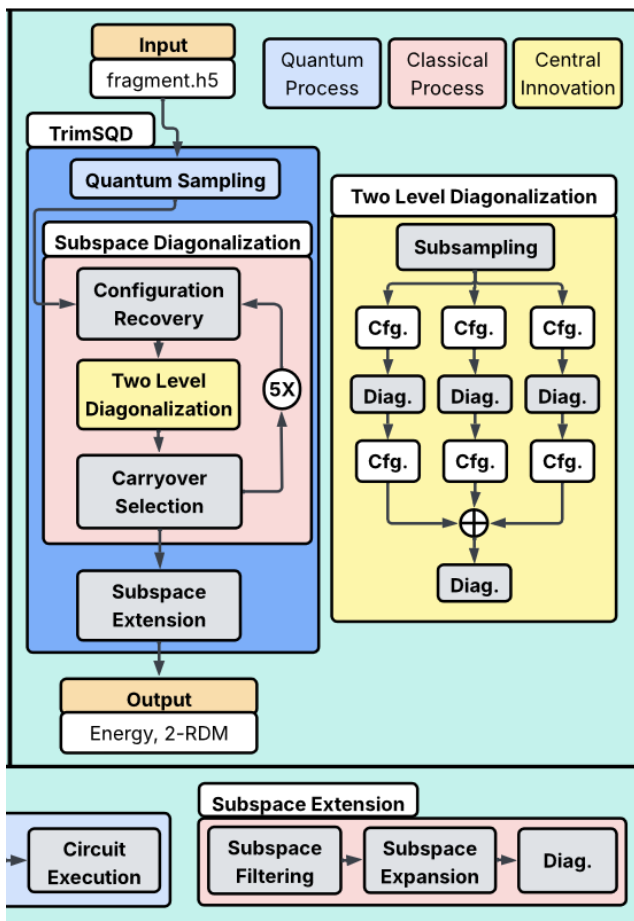
default_shots	: 100000
iterations	: 6
n_batches	: 6
samples_per_batch	: 3000
classical_backend	: python

Method 3: EWF + FCI/SQD – COMPLETED

Fragments	: 10
FCI fragments	: 5
SQD fragments	: 1 (ibm_kingston)
Total energy	: -279.27171716 Ha
Δ from CCSD ref	: -55.4779 mHa



FOLLOW ALONG →



Method 4 — EWF + FCI/TrimSQD

Same workflow as Method 3 — **no additional QPU job**. TrimSQD re-uses the `counts.npy` checkpoint from fragment 2 (C=O) and applies a different classical post-processing policy:

	Standard SQD	TrimSQD
Batches	independent subsamples	one disjoint pool, partitioned
Screening	—	diagonalize each batch, trim top-weight CI strings
Reporting	best-batch energy	merged-subspace diagonalization

TrimSQD trades one extra FCI call per iteration for broader subspace coverage.

This section also computes a CASCI (FCI) reference on the same fragment 2 cluster Hamiltonian state energy for that subspace — so we can benchmark SQD and TrimSQD against it directly.



FOLLOW ALONG →

Re-use QPU samples from SQD

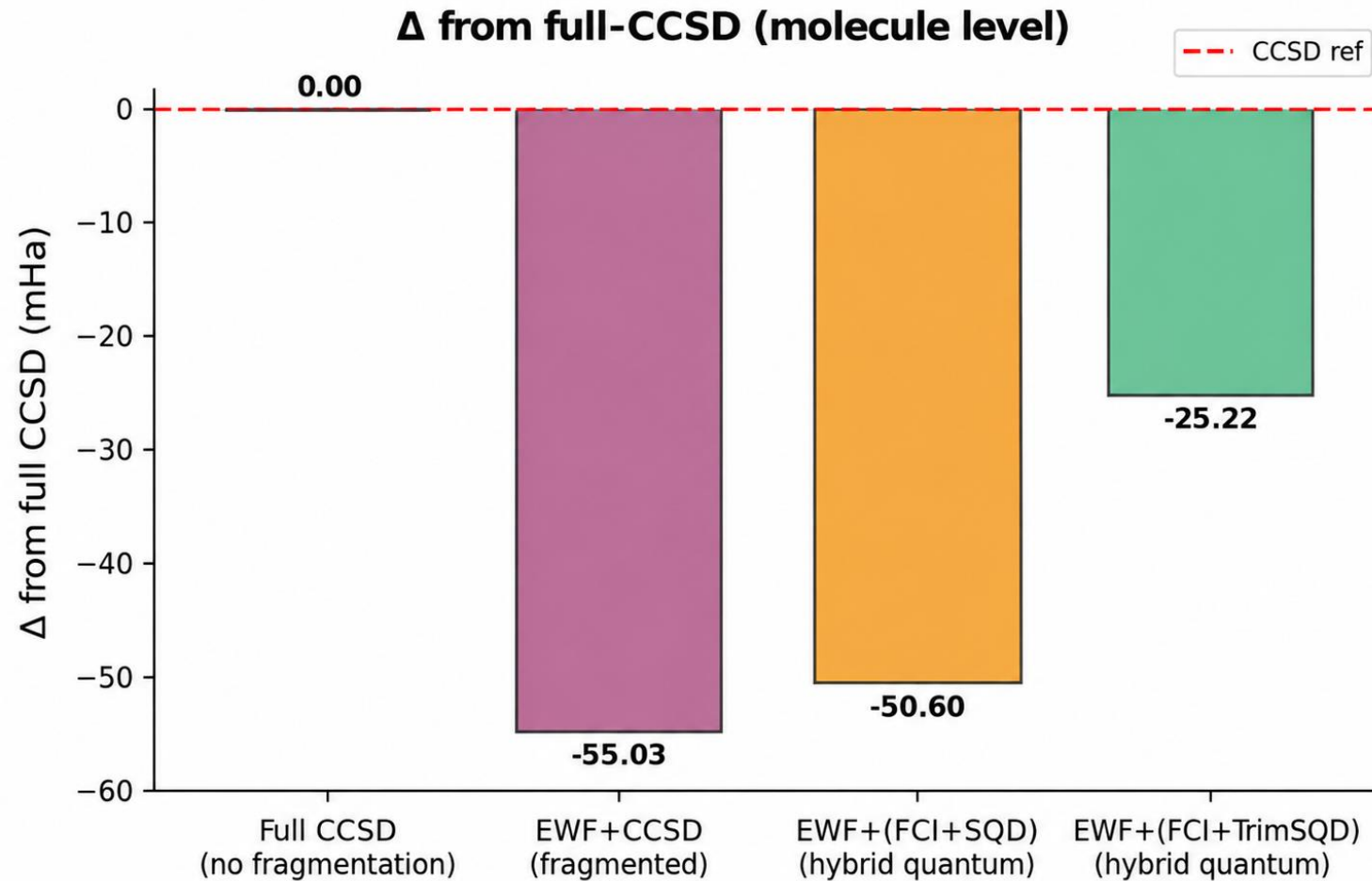
```
16 with _h5py.File(name=_dumpfile, mode='r') as _f:
17     _fg: Group | Dataset | Datatype = _f['fragment_2']
18     _h1e_trim: Group | Dataset | Datatype | Any = _fg['heff'][:]
19     _h2e_trim: Group | Dataset | Datatype | Any = _fg['eris'][:]
20     _norb_trim: int = int(_fg.attrs['norb'])
21     _nocc_trim: int = int(_fg.attrs['nocc'])
22
23     _nelec_trim: tuple[int, int] = (_nocc_trim, _nocc_trim)
24
25     # — Run TrimSQD post-processing —————
26     trim_solver: TrimSQDSolver = TrimSQDSolver(qpu_backend=backend, config=sqd_config)
27     result_trim_frag2: SolverResult = trim_solver._sbd_postprocessing(
28         h1e=_h1e_trim,
29         h2e=_h2e_trim,
30         counts=_counts_raw,
31         norb=_norb_trim,
32         nelec=_nelec_trim,
33         workflow_path=DEMO_DIR / 'results_ewf_adaptive' / 'fragment_2',
34     )
35
36     trim_e: float = result_trim_frag2.energy
```

← The whole point of this slide



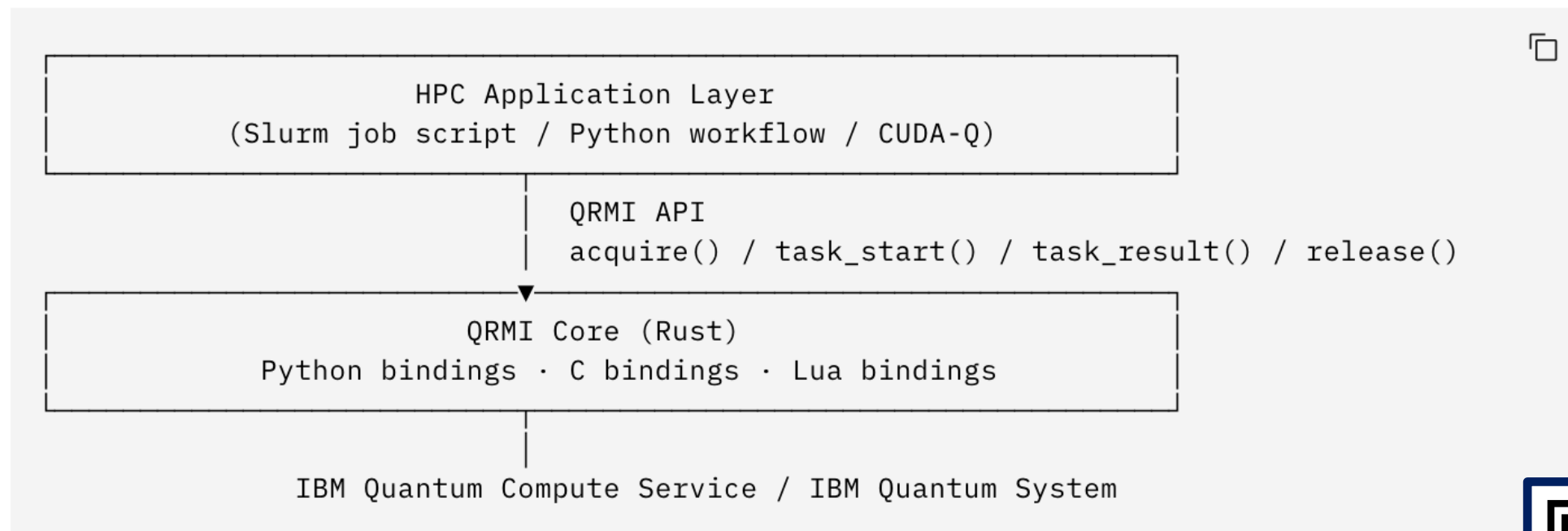
Reconstructed molecular energies

Glycine / STO-3G — SQD vs TrimSQD comparison



What QRMI is

The **Quantum Resource Management Interface (QRMI)** is a middleware library written in Rust that standardizes access to quantum hardware from HPC schedulers and classical applications. It exposes a single unified lifecycle API:



QRMI is published as an open-source project at github.com/qiskit-community/qrmi and is described in the overview paper [arXiv:2506.10052](https://arxiv.org/abs/2506.10052).

[7] Bacher *et al.*, [arXiv:2506.10052](https://arxiv.org/abs/2506.10052) (2025).



Key design choices

Resource lifecycle, not circuit compilation. QRMJ handles the acquire/submit/poll/release lifecycle and nothing else. Circuit compilation, optimization, and transpilation remain in the application layer (for example, Qiskit). This keeps the interface minimal and composable.

Vendor portability model. While QRMJ provides common job-management calls (`acquire` , `task_start` , `task_status` , `task_result` , `release`) across supported hardware backends, changing vendors also requires different compilation passes, vendor-specific payload construction, and result decoding in the application layer.

Native IBM payload format. For IBM Quantum backends, QRMJ uses OpenQASM 3 JSON payloads (`QiskitPrimitive`) conforming to the Qiskit Runtime schema.

Configuration through environment variables. Credentials and endpoint URLs are read from environment variables at runtime. In an HPC cluster, the Slurm QRMJ SPANK plugin sets these automatically when a job is dispatched. In a notebook or interactive session, you load them from a `.env` file. Application code never contains hardcoded credentials or endpoint URLs.

HPC scheduler integration through GRES. When a Slurm job requests quantum resources using the QRMJ SPANK plugin interface (`#SBATCH --gres=qpu:1` and `#SBATCH --qpu=ibm_kingston`), the plugin injects `QRMJ_JOB_QPU_RESOURCES` and `QRMJ_JOB_QPU_TYPES` into the job environment. Applications call `get_job_qpu_resources_and_types()` to discover which resources were allocated — no hardcoded backend names required. `QRMJService` wraps this pattern for Qiskit users.



The core API calls

Call	Purpose
<code>qrmi.acquire()</code>	Acquire access to the resource (for example, opens a dedicated session); returns a lock token
<code>qrmi.target()</code>	Retrieve backend capabilities (qubits, gates, coupling map) as JSON
<code>qrmi.task_start(payload)</code>	Submit a quantum job; returns a job ID
<code>qrmi.task_status(job_id)</code>	Poll job status (<code>Queued</code> , <code>Running</code> , <code>Completed</code> , <code>Failed</code>)
<code>qrmi.task_result(job_id)</code>	Retrieve completed job results as a raw JSON string
<code>qrmi.task_stop(job_id)</code>	Cancel or clean up a job
<code>qrmi.release(lock)</code>	Release the resource lock (for example, closes the session)



Credentials required

- IBM Quantum: IAM API key and Service CRN from [IBM Quantum Platform](#)

For standalone execution, create a `.env` file next to this notebook with the following values, replacing the credential placeholders. Keep this file private. If you select a different backend, update both its name and the environment variable prefixes.

```
ibm_kingston_QRMI_IBM_QCS_ENDPOINT=https://quantum.cloud.ibm.com/api/v1
ibm_kingston_QRMI_IBM_QCS_IAM_ENDPOINT=https://iam.cloud.ibm.com
ibm_kingston_QRMI_IBM_QCS_IAM_APIKEY=<your-iam-api-key>
ibm_kingston_QRMI_IBM_QCS_SERVICE_CRN=<your-crn-starting-with-crn:v1:>
ibm_kingston_QRMI_IBM_QCS_SESSION_MODE=dedicated
ibm_kingston_QRMI_IBM_QCS_SESSION_MAX_TTL=28800
QRMI_JOB_QPU_RESOURCES=ibm_kingston
QRMI_JOB_QPU_TYPES=ibm-quantum-compute-service
```



For a Slurm allocation, use the resource settings and credentials supplied by the cluster. The notebook preserves existing environment values.



HPC context: Slurm resource injection

In an HPC cluster, users request quantum resources using Slurm GRES syntax along with the QRMI SPANK plugin options. The plugin handles credential and resource injection automatically:

```
1 | #SBATCH --gres=qpu:1
2 | #SBATCH --qpu=ibm_kingston
3 | python my_workflow.py # QRMI_JOB_QPU_RESOURCES and QRMI_JOB_QPU_TYPES are already set
```

Application code discovers its allocated resources at runtime — no hardcoded backend names:

```
1 | # get_job_qpu_resources_and_types() reads QRMI_JOB_QPU_RESOURCES / QRMI_JOB_QPU_TYPES
2 | # set by the Slurm SPANK plugin (or manually above in Setup)
3 | qpus, qpu_types = get_job_qpu_resources_and_types()
4 | print("Resources allocated by scheduler:")
5 | for qpu, qpu_type in zip(qpus, qpu_types):
6 |     print(f" {qpu} ({qpu_type})")
7 |
8 | # QRMIService wraps this into a list of ready QuantumResource objects
9 | for r in QRMIService().resources():
10 |     print(
11 |         f"\nQRMIService found: {r.resource_id()} accessible={r.is_accessible()}"
12 |     )
```

Output:

```
Resources allocated by scheduler:
  ibm_kingston (ibm-quantum-compute-service)

QRMIService found: ibm_kingston accessible=True
```



Scaling requires optimizing the workflow and the QPU

- **Decompose the physics**
EWF converts a globally intractable electronic-structure problem into local correlated subproblems.
- **Route the algorithms**
Keep easy fragments classical; reserve quantum resources for the fragments that justify them.
- **Improve the whole workflow**
Embedding, sampling, post-processing, reconstruction, and classical diagonalization all affect the scientific result.
- **Schedule the hardware**
QRMI allows CPU, GPU, and QPU resources to participate in one heterogeneous HPC execution model.

Thank you! To retrieve any material and some additional content from today, please save these links



QRFI Tutorial



Laptop QFM Example



HPC QFM Example

References

- [1] I. Liepuoniute, M. Motta, T. Pellegrini, J. E. Rice, T. P. Gujarati, S. Gil, and G. O. Jones, “Simulation of a Diels–Alder reaction on a quantum computer,” *Phys. Chem. Chem. Phys.* 26, 25181–25191 (2024). DOI: 10.1039/D4CP01314J.
- [2] A. Shajan, D. Kaliakin, F. Liang, T. Pellegrini, H. Doga, S. Bhowmik, S. Das, A. Mezzacapo, M. Motta, and K. M. Merz Jr., “Molecular Quantum Computations on a Protein,” *J. Chem. Theory Comput.*, vol. 22, no. 12, pp. 6041–6056, 2026, doi: 10.1021/acs.jctc.6c00364.
- [3] M. Nusspickel and G. H. Booth, “Systematic Improvability in Quantum Embedding for Real Materials,” *Phys. Rev. X*, vol. 12, 011046 (2022), doi: 10.1103/PhysRevX.12.011046.
- [4] S. Das *et al.*, “Quantum Computations on Fusion Blanket Molten Salts,” arXiv:2606.30402, 2026, doi: 10.48550/arXiv.2606.30402.
- [5] K. M. Merz Jr. *et al.*, “Crossing the 12,000-atom barrier with heterogeneous quantum-classical supercomputing: quantum chemistry of protein-ligand complexes,” arXiv:2605.01138, 2026, doi: 10.48550/arXiv.2605.01138.
- [6] A. Shajan, D. Kaliakin, F. Liang, S. Bhowmik, S. Das, Z. Li, M. Bazayeva, and T. Pellegrini, *Quantum Fragment Methods*, v0.0.1 (2026), GitHub.
- [7] U. Bacher *et al.*, “Quantum resources in resource management systems,” *arXiv preprint arXiv:2506.10052*, 2025.